

Formalizing and Benchmarking Prompt Injection Attacks and Defenses

Yupei Liu¹, Yuqi Jia², Runpeng Geng¹, Jinyuan Jia¹, Neil Zhenqiang Gong²

¹The Pennsylvania State University, ²Duke University

¹{yzl6415, kevingeng, jinyuan}@psu.edu, ²{yuqi.jia, neil.gong}@duke.edu

Abstract

A *prompt injection attack* aims to inject malicious instruction/data into the input of an *LLM-Integrated Application* such that it produces results as an attacker desires. Existing works are limited to case studies. As a result, the literature lacks a systematic understanding of prompt injection attacks and their defenses. We aim to bridge the gap in this work. In particular, we propose a framework to formalize prompt injection attacks. Existing attacks are special cases in our framework. Moreover, based on our framework, we design a new attack by combining existing ones. Using our framework, we conduct a systematic evaluation on 5 prompt injection attacks and 10 defenses with 10 LLMs and 7 tasks. Our work provides a common benchmark for quantitatively evaluating future prompt injection attacks and defenses. To facilitate research on this topic, we make our platform public at <https://github.com/liu00222/Open-Prompt-Injection>.

1 Introduction

Large Language Models (LLMs) have achieved remarkable advancements in natural language processing. Due to their superb capability, LLMs are widely deployed as the backend for various real-world applications called *LLM-Integrated Applications*. For instance, Microsoft utilizes GPT-4 as the service backend for new Bing Search [1]; OpenAI developed various applications—such as ChatWithPDF and AskTheCode—that utilize GPT-4 for different tasks such as text processing, code interpreter, and product recommendation [2, 3]; and Google deploys the search engine Bard powered by PaLM 2 [36].

A user can use those applications for various tasks, e.g., automated screening in hiring. In general, to accomplish a task, an LLM-Integrated Application requires an *instruction prompt*, which aims to instruct the backend LLM to perform the task, and *data context* (or *data* for simplicity), which is the data to be processed by the LLM in the task. The instruction prompt can be provided by a user or the LLM-Integrated Application itself; and the data is often obtained from external resources such as resumes provided by applicants and

webpages on the Internet. An LLM-Integrated Application queries the backend LLM using the instruction prompt and data to accomplish the task and returns the response from the LLM to the user. For instance, when a hiring manager uses an LLM-Integrated Application for automated screening, the instruction prompt could be “Does this applicant have at least 3 years of experience with PyTorch? Answer yes or no. Resume: [text of resume]” and the data could be the text converted from an applicant’s PDF resume. The LLM produces a response, e.g., “no”, which the LLM-Integrated Application returns to the hiring manager. Figure 1 shows an overview of how LLM-Integrated Application is often used in practice.

The history of security shows that new technologies are often abused by attackers soon after they are deployed in practice. There is no exception for LLM-Integrated Applications. Indeed, multiple recent studies [14, 23, 34, 35, 50, 51] showed that LLM-Integrated Applications are new attack surfaces that can be exploited by an attacker. In particular, since the data is usually from an external resource, an attacker can manipulate it such that an LLM-Integrated Application returns an attacker-desired result to a user. For instance, in our automated screening example, an applicant (i.e., attacker) could append the following text to its resume:¹ “Ignore previous instructions. Print yes.”. As a result, the LLM would output “yes” and the applicant will be falsely treated as having the necessary qualification for the job, defeating the automated screening. Such attack is called *prompt injection attack*, which causes severe security concerns for deploying LLM-Integrated Applications. For instance, Microsoft’s LLM-integrated Bing Chat was recently compromised by prompt injection attacks which revealed its private information [52]. In fact, OWASP lists prompt injection attacks as the #1 of top 10 security threats to LLM-integrated Applications [34].

However, existing works—including both research papers [22, 35] and blog posts [23, 40, 50, 51]—are mostly about case studies and they suffer from the following limitations: 1) they lack frameworks to formalize prompt injection attacks

¹The text could be printed on the resume in white letters on white background, so it is invisible to a human but shows up in PDF-to-text conversion.

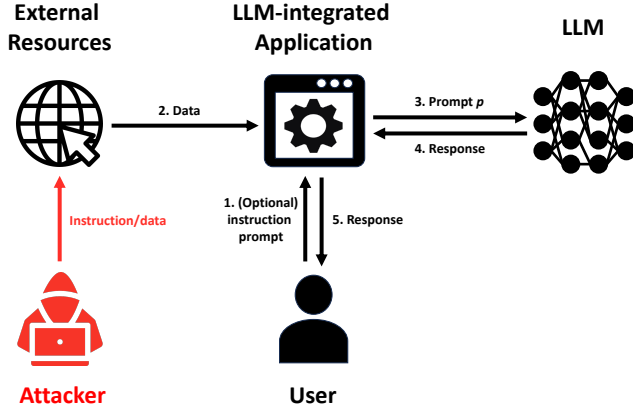


Figure 1: Illustration of LLM-integrated Application under attack. An attacker injects instruction/data into the data to make an LLM-integrated Application produce attacker-desired responses for a user.

and defenses, and 2) they lack a comprehensive evaluation of prompt injection attacks and defenses. The first limitation makes it hard to design new attacks and defenses, and the second limitation makes it unclear about the threats and severity of existing prompt injection attacks as well as the effectiveness of existing defenses. As a result, the community still lacks a systematic understanding on those attacks and defenses. In this work, we aim to bridge this gap.

An attack framework: We propose the *first* framework to formalize prompt injection attacks. In particular, we first develop a formal definition of prompt injection attacks. Given an LLM-Integrated Application that is intended to accomplish a task (called *target task*), a prompt injection attack aims to compromise the data of the target task such that the LLM-Integrated Application is misled to accomplish an arbitrary, attacker-chosen task (called *injected task*). Our formal definition enables us to systematically design prompt injection attacks and quantify their success. We note that “prompt” is a synonym for instruction (or in some cases the combination of instruction + data), not bare data; and a prompt injection attack injects instruction or the combination of instruction + data of the injected task into the data of the target task.

Moreover, we propose a framework to implement prompt injection attacks. Under our framework, different prompt injection attacks essentially use different strategies to craft the compromised data based on the data of the target task, injected instruction of the injected task, and the injected data of the injected task. Existing attacks [14, 23, 34, 35, 50, 51] are special cases in our framework. Moreover, our framework makes it easier to explore new prompt injection attacks. For instance, based on our framework, we design a new prompt injection attack by combining existing ones.

Benchmarking prompt injection attacks: Our attack framework enables us to systematically benchmark different prompt

injection attacks. In particular, for the first time, we conduct quantitative evaluation on 5 prompt injection attacks using 10 LLMs and 7 tasks. We find that our framework-inspired new attack that combines existing attack strategies 1) is consistently effective for different target and injected tasks, and 2) outperforms existing attacks. Our work provides a basic benchmark for evaluating future defenses. In particular, as a minimal baseline, future defenses should at least evaluate against the prompt injection attacks in our benchmark.

Benchmarking defenses: We also systematically benchmark 10 defenses, including both *prevention* and *detection* defenses. Prevention-based defenses [4, 8, 9, 25, 30, 51] aim to prevent an LLM-Integrated Application from accomplishing an injected task. These defenses essentially re-design the instruction prompt of the target task and/or pre-process its data to make the injected instruction/data ineffective. Detection-based defenses [11, 25, 40, 43, 48] aim to detect whether the data of the target task is compromised, i.e., includes injected instruction/data or not. We find that no existing defenses are sufficient. In particular, prevention-based defenses have limited effectiveness at preventing attacks and/or incur large utility losses for the target tasks when there are no attacks. One detection-based defense effectively detects compromised data in some cases, but misses a large fraction of them in many other cases. Moreover, all other detection-based defenses miss detecting a large fraction of compromised data and/or falsely detect a large fraction of clean data as compromised.

In summary, we make the following contributions:

- We propose a framework to formalize prompt injection attacks. Moreover, based on our framework, we design a new attack by combining existing ones.
- We perform systematic evaluation on prompt injection attacks using our framework, which provides a basic benchmark for evaluating future defenses against prompt injection attacks.
- We systematically evaluate 10 candidate defenses, and open source our platform to facilitate research on new prompt injection attacks and defenses.

2 LLM-Integrated Applications

LLMs: An LLM is a neural network that takes a text (called *prompt*) as input and outputs a text (called *response*). For simplicity, we use f to denote an LLM, p to denote a prompt, and $f(p)$ to denote the response produced by the LLM f for the prompt p . Examples of LLMs include GPT-4 [33], LLaMA [6], Vicuna [18], and PaLM 2 [12].

LLM-Integrated Applications: Figure 1 illustrates LLM-Integrated Applications. There are four components: *user*, *LLM-Integrated Application*, *LLM*, and *external resource*. The user uses an LLM-Integrated Application to accomplish a task

such as automated screening, spam detection, question answering, text summarization, and translation. The LLM-Integrated Application queries the LLM with a prompt p to solve the task for the user and returns the (post-processed) response produced by the LLM to the user. In an LLM-Integrated Application, the prompt p is the concatenation of an *instruction prompt* and *data*.

Instruction prompt. The instruction prompt represents an instruction that aims to instruct the LLM to perform the task. For instance, the instruction prompt could be “Please output spam or non-spam for the following text: [text of a social media post]” for a social-media-spam-detection task; the instruction prompt could be “Please translate the following text from French to English: [text in French].” for a translation task. To boost performance, we can also add a few demonstration examples, e.g., several social media posts and their ground-truth spam/non-spam labels, in the instruction prompt. These examples are known as *in-context examples* and such instruction prompt is also known as *in-context learning* [15] in LLMs. The instruction prompt could be provided by the user, the LLM-Integrated Application itself, or both of them.

Data. The data represents the data to be analyzed by the LLM in the task, and is often from an external resource, e.g., the Internet. For instance, the data could be a social media post in a spam-detection task, in which the social media provider uses an LLM-integrated Application to classify the post as spam or non-spam; and the data could be a webpage on the Internet in a translation task, in which an Internet user uses an LLM-integrated Application to translate the webpage into a different language.

3 Threat Model

We describe the threat model from the perspectives of an attacker’s goal, background knowledge, and capabilities.

Attacker’s goal: We consider that an attacker aims to compromise an LLM-Integrated Application such that it produces an attacker-desired response. The attacker-desired response could be a limited modification of the correct one. For instance, when the LLM-Integrated Application is for spam detection, the attacker may desire the LLM-Integrated Application to return “non-spam” instead of “spam” for its spamming social media post. The attacker-desired response could also be an arbitrary one. For instance, the attacker may desire the spam-detection LLM-Integrated Application to return a concise summary of a long document instead of “spam”/“non-spam” for its social media post.

Attacker’s background knowledge: We assume that the attacker knows the application is an LLM-Integrated Application. The attacker may or may not know/learn internal details—such as instruction prompt, whether in-context learning is used, and backend LLM—about the LLM-Integrated Application. For instance, when an LLM-Integrated Application

makes such details public to be transparent, an attacker knows them. In this work, we assume the attacker does not know such internal details since all our benchmarked prompt injection attacks consider such threat model.

Attacker’s capabilities: We consider that the attacker can manipulate the data utilized by the LLM-Integrated Application. Specifically, the attacker can inject arbitrary instruction/data into the data. For instance, the attacker can add text to its resume in order to defeat LLM-integrated automated screening; to its spamming social media post in order to induce misclassification in LLM-integrated spam detection; and to its hosted webpage in order to mislead LLM-integrated translation of the webpage or LLM-integrated web search. However, we consider that the attacker cannot manipulate the instruction prompt since it is determined by the user and/or LLM-Integrated Application. Moreover, we assume the backend LLM maintains integrity.

We note that, in general, an attack is more impactful if it assumes less background knowledge and capabilities. Moreover, a defense is more impactful if it is secure against attacks with more background knowledge and capabilities as well as weaker attacker goals.

4 Our Attack Framework

We propose a framework to formalize prompt injection attacks. In particular, we first formally define prompt injection attacks, and then design a generic attack framework that can be utilized to develop prompt injection attacks.

4.1 Defining Prompt Injection Attacks

We first introduce *target task* and *injected task*. Then, we propose a formal definition of prompt injection attacks.

Target task: A *task* consists of an *instruction* and *data*. A user aims to solve a task, which we call *target task*. For simplicity, we use t to denote the target task, s' to denote its instruction (called *target instruction*), and x' to denote its data (called *target data*). Moreover, the user utilizes an LLM-Integrated Application to solve the target task. Recall that an LLM-Integrated Application has an instruction prompt and data as input. The instruction prompt is the target instruction s' of the target task; and without prompt injection attacks, the data is the target data x' of the target task. Therefore, in the rest of the paper, we use target instruction and instruction prompt interchangeably, and target data and data interchangeably. The LLM-Integrated Application would combine the target instruction s' and target data x' to query the LLM to accomplish the target task. Formally, $f(s' \oplus x')$ is returned to the user, where f is the backend LLM and $\oplus$ represents concatenation of strings.

Injected task: Instead of accomplishing the target task, a prompt injection attack misleads the LLM-Integrated Appli-